

Universidade Tecnológica Federal do Paraná (UTFPR)
Sistemas Operacionais — 2026.2
Docente: Prof. Dr. Marco Aurélio Wehrmeister

Telas do Simulador

Simulador de SO Multitarefa de Tempo Compartilhado — Projeto A

1. Identificação da equipe

Trabalho desenvolvido em dupla:

- Ana Carolina Czelen Lipe
- Isabela Bella Bortoleto

2. Como a informação chega à tela

Este documento apresenta todas as telas do simulador, a explicação de cada elemento visual e de onde vem a informação mostrada em cada um.

O simulador é um único programa em C (`simulador.exe`). Ao ser aberto, ele inicia um servidor HTTP na própria máquina (`127.0.0.1:8080`) e abre o navegador. A página não calcula nada da simulação: ela pede os dados ao programa em C e só os exibe. O caminho de uma informação é sempre o mesmo:

1. o arquivo de configuração (`.txt`) é escolhido na página e enviado ao C, que o lê em `config.c` (`config_carregar_texto`);
2. os dados de cada tarefa ficam no TCB (`tcb.h`), e a simulação, tick a tick, é feita pelo núcleo (`kernel.c`), que guarda uma cópia do estado a cada tick (o histórico);
3. o servidor (`servidor.c`) responde aos pedidos da página. O estado atual vai em formato JSON (`estado_json.c`), e o gráfico de Gantt vai pronto, como imagem SVG (`gantt.c`);
4. a página (`web/index.html`, `web/estilo.css` e `web/app.js`) recebe essas respostas e preenche cada elemento.

Nas tabelas a seguir, "JSON do estado" é a resposta das rotas `/api/estado`, `/api/carregar`, `/api/avancar`, `/api/retroceder`, `/api/executar_tudo`, `/api/editar` e `/api/algoritmo`: todas devolvem o estado do tick atual, montado pela função `estado_json_atual` (`estado_json.c`) a partir de `s->atual`.

3. Tela inicial

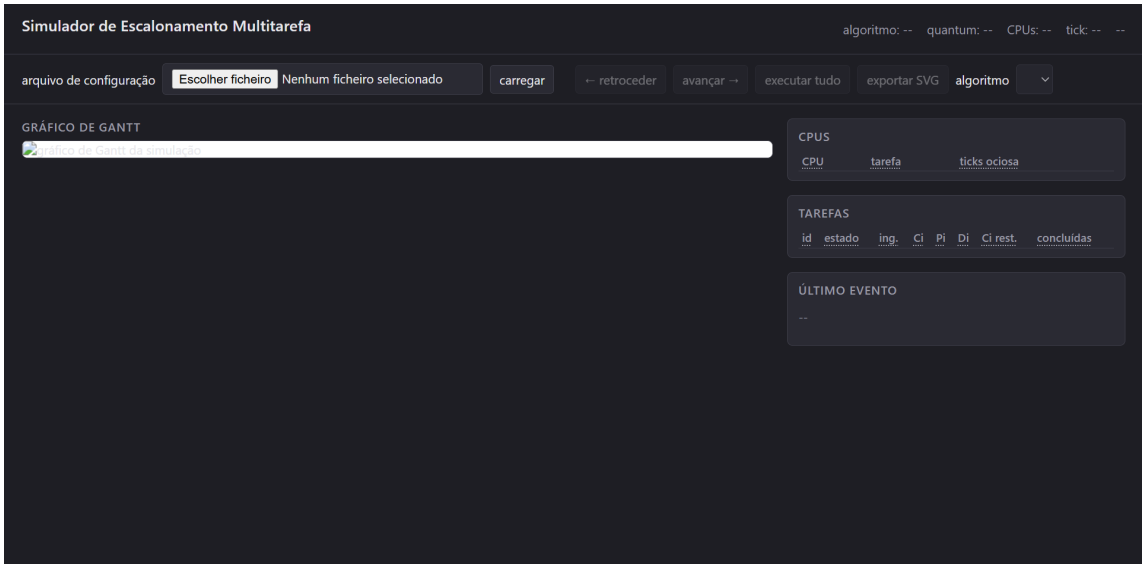


Figura 1: Tela inicial, antes de carregar um arquivo de configuração

Elemento	O que mostra	De onde vem a informação
Título	"Simulador de Escalonamento Multitarefa"	texto fixo do `index.html`
Cabeçalho (algoritmo, quantum, CPUs, tick, situação)	"–" em todos os campos: nenhuma simulação carregada	JSON do estado com `"carregado": false`; a página só preenche o cabeçalho quando há simulação (`aoAbrirPagina`, em `app.js`)
Arquivo de configuração (seletor e nome)	botão do próprio navegador para escolher o arquivo e o nome do escolhido	seletor de arquivos do sistema operacional; permite qualquer pasta, inclusive um *pendrive*. O C recebe o **conteúdo** do arquivo, nunca um caminho
Botão "carregar"	envia o arquivo escolhido ao simulador	`aoCarregar` (`app.js`) → rota `/api/carregar` → `rota_carregar` (`servidor.c`) → `config_carregar_texto` e `sim_iniciar`. Enquanto espera, mostra "carregando..."
Botões retroceder, avançar, executar tudo, exportar SVG	desativados (cinza)	só são ativados depois que um arquivo é carregado (`definirControlesHabilitados`)

Elemento	O que mostra	De onde vem a informação
Seletor “algoritmo”	desativado e vazio	as opções chegam com o primeiro estado carregado (ver Seção 4)

4. Tela da simulação

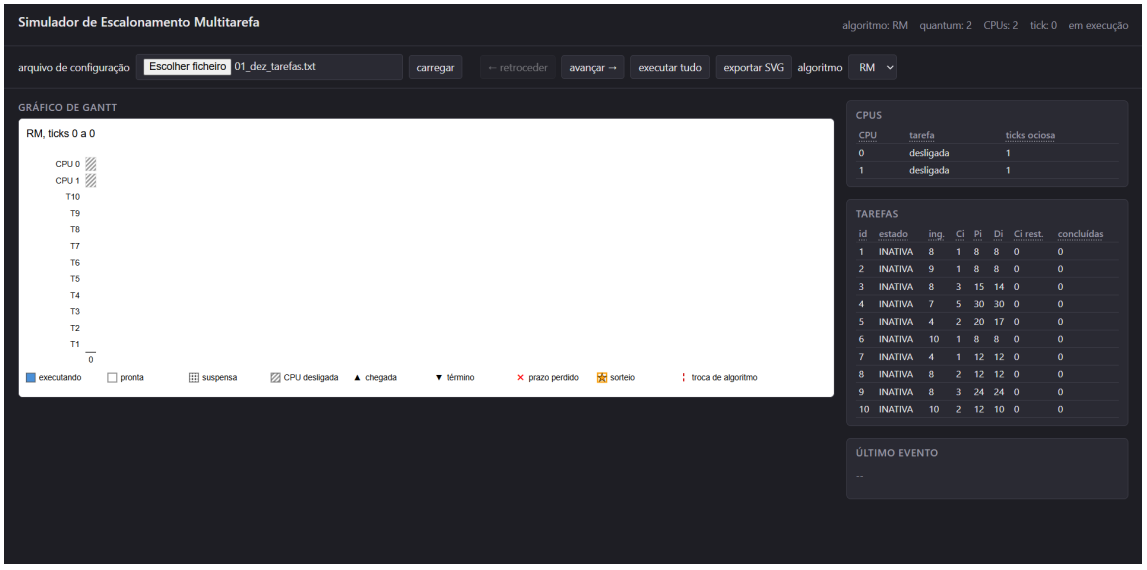


Figura 2: Tela da simulação, com o arquivo `01_dez_tarefas.txt` carregado

4.1. Cabeçalho e barra de controles

Elemento	O que mostra	De onde vem a informação
algoritmo	algoritmo de escalonamento em vigor (RM ou EDF)	1º campo da 1ª linha do arquivo, lido em `config.c`; pode mudar pelo seletor “algoritmo”. Vem no campo `"algoritmo"` do JSON do estado
quantum	tempo máximo seguido que uma tarefa pode usar a CPU (0 = sem limite)	2º campo da 1ª linha do arquivo; campo `"quantum"` do JSON
CPUs	quantidade de processadores	3º campo da 1ª linha do arquivo; campo `"ncpus"` do JSON
tick	instante atual do relógio global	campo `tick` do estado, incrementado a cada `sim_avancar` (`kernel.c`)
situação	“em execução” ou “concluída”	campo `"terminada"` do JSON: a simulação termina quando todas as tarefas completam 10 execuções (`todas_concluidas`)

Elemento	O que mostra	De onde vem a informação
retroceder	volta um tick; desativado no tick 0	rota `/api/retroceder` → `sim_retroceder`, que restaura a foto anterior do histórico
avançar	executa um tick (modo passo a passo)	rota `/api/avancar` → `sim_avancar`
executar tudo	executa até o fim (modo completo)	rota `/api/executar_tudo` → `sim_executar_tudo`, que repete o `sim_avancar` até a simulação terminar
exportar SVG	baixa a imagem do Gantt da simulação inteira	rota `/api/exportar_svg`: a mesma `gantt_svg` da tela, do tick 0 ao tick atual (Seção 8)
Seletor "algoritmo"	algoritmos disponíveis e o que está em vigor; trocar no meio da simulação vale a partir do próximo tick	as opções vêm do campo `"algoritmos"` do JSON: a lista dos escalonadores registrados em `escalonador.c`, então um algoritmo novo aparece sozinho. A troca usa a rota `/api/algoritmo` → `sim_trocar_algoritmo`

4.2. Tabela de CPUs

Elemento	O que mostra	De onde vem a informação
CPU	número do processador	posição do processador no vetor de CPUs do estado (campo `id` da struct `CPU`, `tcb.h`)
tarefa	tarefa que ocupa a CPU neste tick, ou "desligada"	decidido pela `escalonar` (`kernel.c`), com a fila de prontos global e o algoritmo em vigor. O C guarda a posição da tarefa e converte para o ID em `sai_cpu` (`estado_json.c`)
ticks ociosa	quantos ticks esta CPU já passou desligada, desde o início da simulação	campo `ticks_desligada`, somado na `escalonar` a cada tick em que a CPU fica sem tarefa

4.3. Tabela de tarefas

Uma linha por tarefa, com o que o TCB guarda dela no tick atual. Os títulos curtos seguem a notação do capítulo de tempo real (C_i, P_i, D_i); ao passar o mouse sobre um título, aparece o nome completo. Clicar numa linha abre o painel de edição (Seção 7).

Elemento	O que mostra	De onde vem a informação
id	identificador da tarefa	1º campo da linha da tarefa no arquivo
estado	INATIVA, PRONTA, EXECUTANDO, SUSPENSA ou CONCLUIDA	campo `estado` do TCB, mudado pelo núcleo: `ativar_tarefa` (chegada), `escalonar` (CPU), `executar` (término) e pela edição manual; o texto vem de `estado_nome` (`tcb.c`)
ing.	ingresso: tick em que a tarefa entra no sistema	3º campo da linha da tarefa no arquivo
C_i	tempo de computação: ticks de CPU que cada execução precisa	4º campo do arquivo (`duracao`); pode ser editado e vale a partir da próxima chegada
P_i	período: a cada quantos ticks chega uma execução nova	5º campo do arquivo. Período 0 é tarefa aperiódica, ignorada com aviso no Projeto A
D_i	prazo (*deadline*) relativo: em quantos ticks, contados da chegada, cada execução tem de terminar	6º campo do arquivo; em branco, vale o período. Pode ser editado
Ci rest.	ticks de CPU que faltam para a execução atual terminar	campo `exec_restante` do TCB: começa igual a C_i em cada chegada e diminui 1 por tick na CPU (`executar`)
concluídas	execuções (instâncias) já concluídas	campo `ativacoes` do TCB, somado na `executar` a cada término; na 10ª a tarefa vira CONCLUIDA
Linha destacada	a tarefa aberta no painel de edição	gerado pela página (`atualizarTabelaTarefas`, `app.js`)

4.4. Último evento

Elemento	O que mostra	De onde vem a informação
Texto	o último acontecimento relevante, por exemplo “simulação concluída no tick 110”, “algoritmo trocado para EDF no tick 10” ou “simulação interrompida no limite de 5000 ticks”	campo `ultimo_evento` da simulação, escrito pelo núcleo (`kernel.c`) nas funções de avançar, retroceder e trocar o algoritmo

5. Gráfico de Gantt

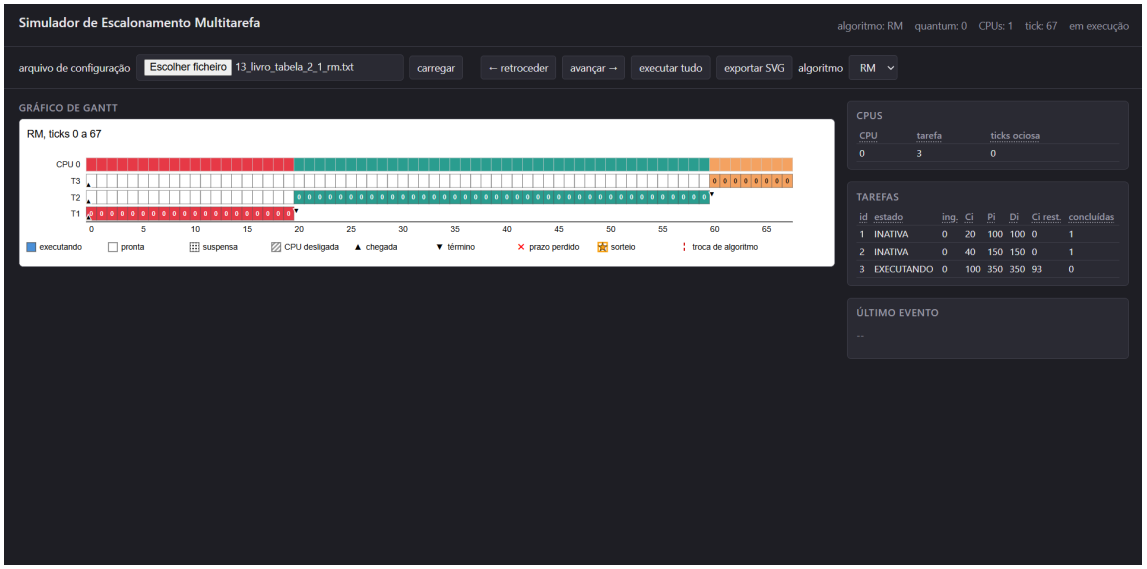


Figura 3: Gráfico de Gantt com chegadas, términos e perdas de prazo

O gráfico é desenhado inteiramente no programa em C, pela função `gantt_svg` (`gantt.c`), a partir do histórico: a cópia do estado guardada a cada tick. A página só exibe a imagem SVG pronta, pedida à rota `/api/gantt` a cada passo. Cada coluna do gráfico é um tick; cada célula mostra o estado daquela tarefa naquele tick.

Elemento	**O que mostra**	**De onde vem a informação**
Título	algoritmo e intervalo de ticks mostrado (“RM, ticks 0 a 30”)	algoritmo da foto do primeiro tick do intervalo. Na tela aparecem os últimos ticks (todos, em arquivos pequenos); o arquivo exportado tem todos
Faixa das CPUs (CPU 0, CPU 1, ...)	em cada tick, a cor da tarefa que ocupa aquela CPU; hachurado cinza quando a CPU está desligada	campo `tarefa` de cada CPU, em cada foto do histórico
Linhas das tarefas (T1, T2, ...)	uma linha por tarefa, com o menor ID embaixo, junto do eixo do tempo	`ordem_por_id` (`gantt.c`)
Bloco colorido	tarefa executando, na cor da tarefa	estado EXECUTANDO na foto daquele tick; a cor é o 2º campo do arquivo (formato RRGGBB)
Número dentro do bloco	em qual CPU a tarefa executou naquele tick	campo `cpu` do TCB, na foto daquele tick

Elemento	**O que mostra**	**De onde vem a informação**
Célula só com contorno	tarefa pronta, esperando CPU: ausência de cor	estado PRONTA na foto daquele tick
Célula pontilhada preta	tarefa suspensa	estado SUSPENSA. No Projeto A só acontece por edição manual, porque os eventos que suspendem uma tarefa são do Projeto B
▲ chegada	chegada de uma execução nova, no instante exato	`marca_em` (`gantt.c`): o campo `ativacao` do TCB igual ao tick, ou o contador de execuções pendentes aumentou (chegada de uma tarefa ainda atrasada)
▼ término	término de uma execução, no instante exato	`marca_em`: o contador `ativacoes` aumentou em relação ao tick anterior
× prazo perdido	uma execução passou do prazo absoluto sem terminar	`marca_em`: o tick em que vence o prazo absoluto (`deadline_abs`, chegada + D_i), marcado pela `verificar_prazos` (`kernel.c`). Cada execução atrasada tem o seu, inclusive as que esperam na fila
★ e borda amarela: sorteio	a tarefa ganhou a CPU num desempate por sorteio	campo `sorteada` do TCB, marcado pelo `escalonador_ordenar` (`escalonador.c`) quando o empate passa por todos os critérios
Linha vermelha tracejada	troca de algoritmo durante a simulação: à esquerda, o algoritmo antigo; à direita, o novo	o nome do algoritmo muda de uma foto para a seguinte
Eixo do tempo	o tick real, a cada 5	posição de cada coluna no histórico
Legenda	o significado de cada elemento acima	desenhada pela própria `gantt_svg`, dentro da imagem, para ficar igual na tela e no arquivo exportado

6. Mensagens de erro e de aviso

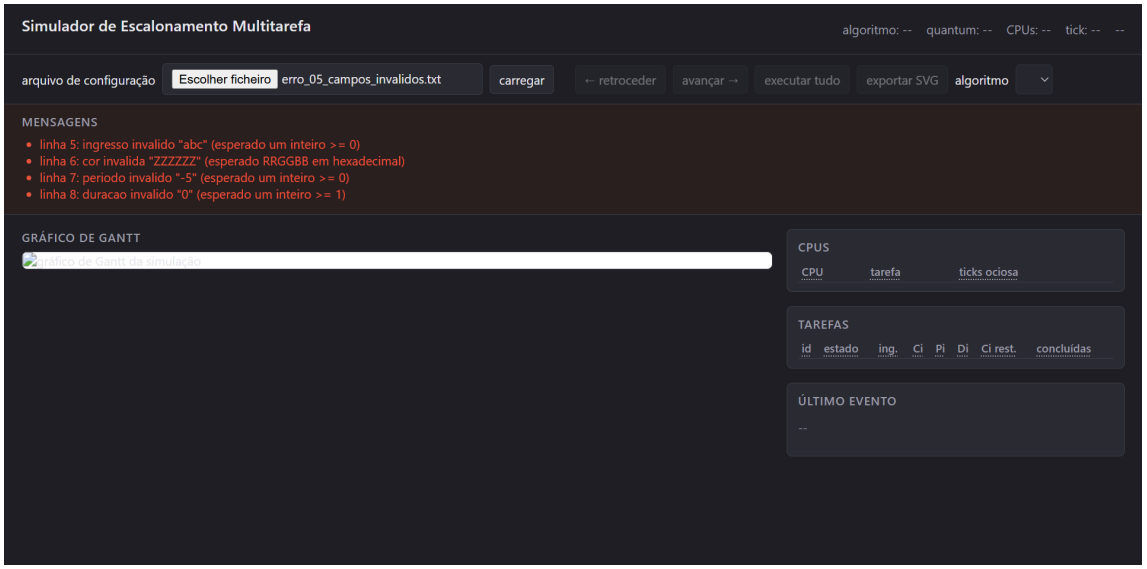


Figura 4: Erros do arquivo `erro_05_campos_invalidos.txt`, cada um com a sua linha

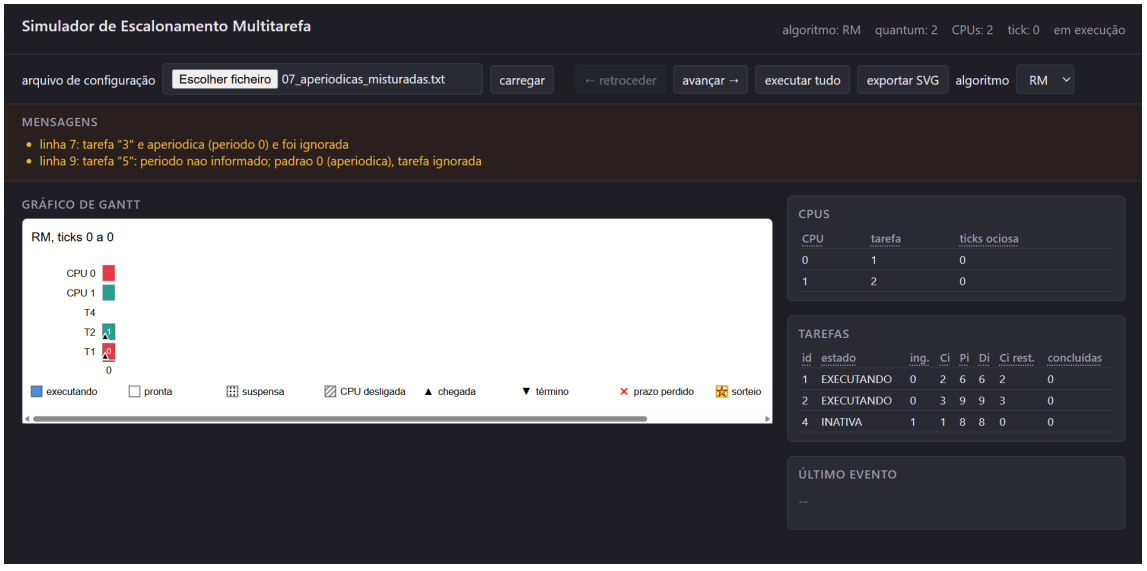


Figura 5: Avisos do arquivo `07_aperiodicas_misturadas.txt`: tarefas aperiódicas ignoradas

Elemento	O que mostra	De onde vem a informação
Área "mensagens"	aparece só quando há algo a dizer	`mostrarDiagnostico` (`app.js`)
Erros (vermelho)	por que o arquivo não pôde ser carregado, com o número da linha; todos de uma vez	campo `"erros"` da resposta de `/api/carregar`: as mensagens de `diag_erro` (`config.c`) e de `sim_iniciar` (ex.: algoritmo inexistente). Arquivo que não é texto é recusado antes, por `parece_texto` (`servidor.c`)
Avisos (amarelo)	o arquivo carregou, mas algo foi ajustado, por exemplo uma tarefa aperiódica ignorada	campo `"avisos"`: as mensagens de `diag_aviso` (`config.c`)
Sem resposta do simulador	o programa em C foi fechado ou não respondeu	gerada pela própria página, em `chamarApi` (`app.js`), quando o pedido ao servidor falha
Arquivo grande demais	arquivo acima de 1 MB	gerada pela página antes de enviar (`aoCarregar`)

7. Painel de edição de tarefa

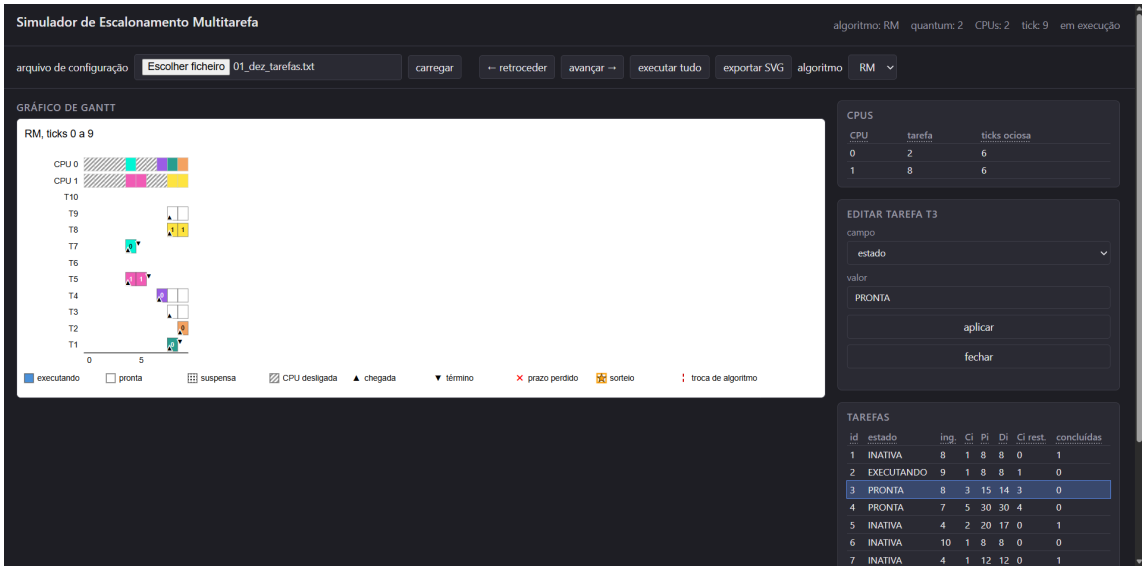


Figura 6: Painel de edição aberto para a tarefa T3

Permite mudar o estado ou uma característica de qualquer tarefa, em qualquer passo da simulação. Quem valida é o núcleo, em `sim_editar_tarefa` (`kernel.c`); a página só envia o pedido (rota `/api/editar`) e mostra a resposta.

Elemento	O que mostra	De onde vem a informação
Título “editar tarefa T3”	a tarefa que está sendo editada	a linha clicada na tabela de tarefas (`abrirInspetor`, `app.js`)
campo	o que editar: estado, C_i (tempo de computação), Ci rest. (tempo restante) ou D_i (prazo), na notação do livro	opções fixas do `index.html`, que são os campos aceitos pelo `sim_editar_tarefa`
valor	o valor novo; o painel já abre com o valor atual	o valor sugerido é o campo da tarefa no JSON do estado (`sugerirValor`, `app.js`)
aplicar	envia a edição	rota `/api/editar` → `rota_editar` (`servidor.c`) → `sim_editar_tarefa`
fechar	fecha o painel	página (`fecharInspetor`)
Mensagem verde	“alteração aplicada no tick N”	gerada pela página quando o C aceita a edição
Mensagem vermelha	por que a edição foi recusada, por exemplo “a tarefa 2 não está ativa; não há o que suspender”	o motivo escrito pelo `sim_editar_tarefa`, enviado no campo `"erros"` da resposta

8. Imagem exportada

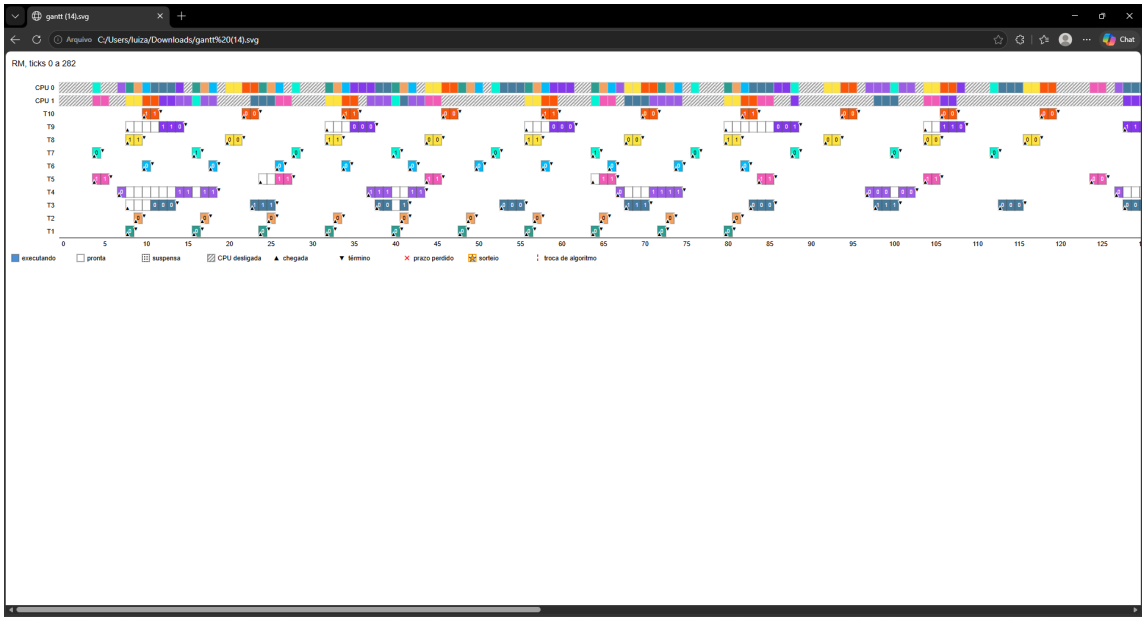


Figura 7: Arquivo `gantt.svg` exportado, aberto fora do simulador

O botão “exportar SVG” baixa o arquivo `gantt.svg`. Ele é gerado pela mesma função da tela, `gantt_svg`, com os mesmos elementos da Seção 5, mas cobrindo

todos os ticks do histórico, do 0 ao tick atual. Não é uma captura de tela: é um desenho vetorial escrito pelo programa a partir dos dados da simulação, que pode ser ampliado sem perder a nitidez e que não depende do tamanho da janela. Para a imagem final, a simulação deve ser executada até o fim antes de exportar.